

Robust & Standard I/O

Outline

- Robust I/O
- Standard I/O
- Suggested Reading
 - 10.5, 10.10-10.11

The RIO Package

- RIO is a set of wrappers that provide efficient and **robust I/O** in apps, such as network programs that are subject to **short counts**
- RIO provides two different kinds of functions
 - Unbuffered input and output of binary data
 - `rio_readn` and `rio_writen`
 - Buffered input of binary data and text lines
 - `rio_readlineb` and `rio_readnb`

Robust I/O

- Unbuffered Input and Output
 - Transfer data directly between memory and a file, with **no application-level buffering**

```
#include "csapp.h"
ssize_t rio_readn(int fd, void *usrbuf, size_t count);
ssize_t rio_writen(int fd, void *usrbuf, size_t count);
    return: number of bytes read (0 if EOF) or written,
            -1 on error
```

```
1  ssize_t rio_readn(int fd, void *buf, size_t count)
2  {
3      size_t nleft = count;
4      ssize_t nread;
5      char *ptr = buf;
6
7      while (nleft > 0) {
8          if ((nread = read(fd, ptr, nleft)) < 0) {
9              if (errno == EINTR)
10                 nread = 0; /* and call read() again */
11                 else
12                     return -1; /* errno set by read() */
13             }
14             else if (nread == 0)
15                 break; /* EOF */
16             nleft -= nread;
17             ptr += nread;
18         }
19         return (count - nleft); /* return >= 0 */
20     }
```

```

1  ssize_t rio_writen(int fd, const void *buf, size_t count)
2  {
3      size_t nleft = count;
4      ssize_t nwritten;
5      const char *ptr = buf;
6
7      while (nleft > 0) {
8          if ((nwritten = write(fd, ptr, nleft)) <= 0) {
9              if (errno == EINTR)
10                 nwritten = 0; /* and call write() again */
11                 else
12                     return -1; /* errno set by write() */
13             }
14             nleft -= nwritten;
15             ptr += nwritten;
16         }
17         return count;
18     }

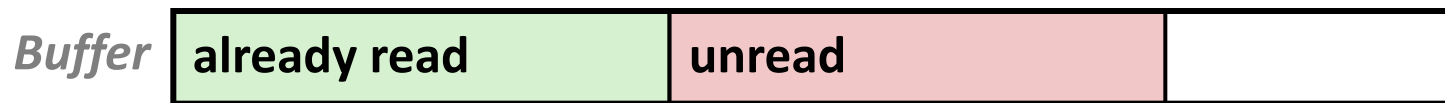
```

Buffered I/O: Motivation

- Applications often read/write one character at a time
 - `getc`, `putc`, `ungetc`
 - `gets`, `fgets`
 - Read line of text one character at a time, stopping at newline
- Implementing as Unix I/O calls expensive
 - `read` and `write` require Unix kernel calls
 - > 10,000 clock cycles

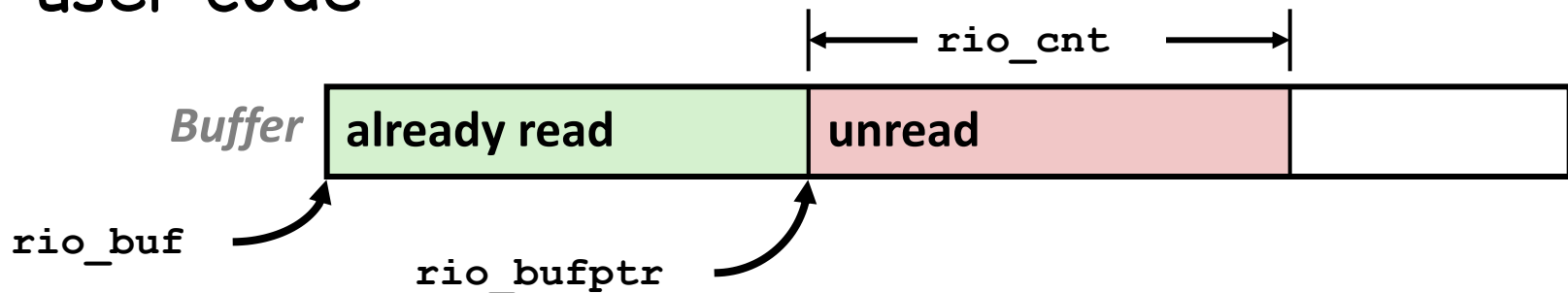
Buffered I/O: Motivation

- Solution: Buffered read
 - Use Unix `read` to grab block of bytes
 - User input functions take one byte at a time from buffer
 - Refill buffer when empty



Buffered I/O: Implementation

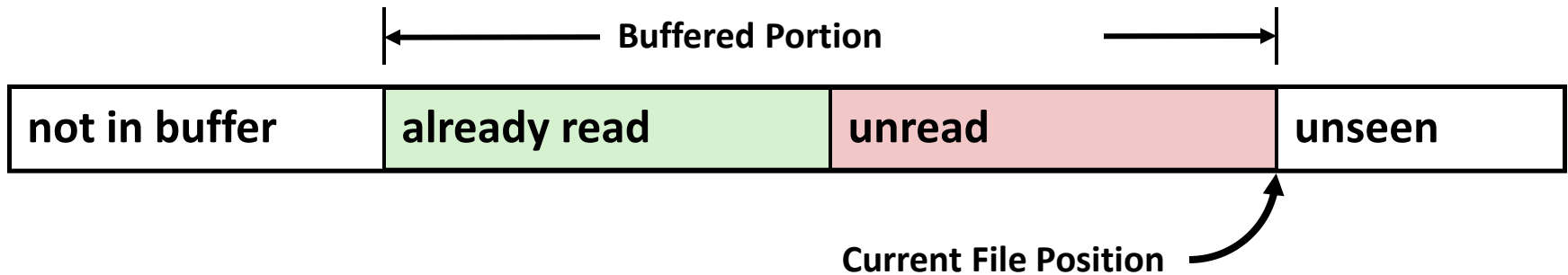
- For reading from file
- File has associated buffer to hold bytes that have been read from file but not yet read by user code



```
typedef struct {  
    int rio_fd;           /* descriptor for this internal buf */  
    int rio_cnt;          /* unread bytes in internal buf */  
    char *rio_bufptr;     /* next unread byte in internal buf */  
    char rio_buf[RIO_BUFSIZE]; /* internal buffer */  
} rio_t;
```

Buffered I/O: Implementation

- Layered on Unix file:



Robust I/O

- Buffered Input and Output
 - Efficiently read text lines and binary data from a file whose contents are cached in an **application-level buffer**

```
#include "csapp.h"
void rio_readinitb(rio_t *rp, int fd) ;
ssize_t rio_readlineb(rio_t *rp,
                      void *usrbuf, size_t maxlen);
ssize_t rio_readnb(rio_t *rp,
                   void *usrbuf, size_t maxlen);
returns: number of bytes read (0 if EOF), -1 on error
```

Robust I/O

```
#define RIO_BUFSIZE 8192
typedef struct {
    int rio_fd;
    int rio_cnt;
    char *rio_bufptr;
    char rio_buf[RIO_BUFSIZE];
} rio_t ;

void rio_readinitb(rio_t *rp, int fd)
{
    rp->rio_fd = fd ;
    rp->rio_cnt = 0 ;
    rp->rio_bufptr = rp->rio_buf ;
}
```

Robust I/O

```
#include "csapp.h"

int main(int argc, char **argv)
{
    int n;
    rio_t rio;
    char buf[MAXLINE];

    Rio_readinitb(&rio, STDIN_FILENO);
    while ((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0)
        Rio_writen(STDOUT_FILENO, buf, n);
}
```

```
1 static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n)
2 {
3     int cnt = 0;
4
5     while (rp->rio_cnt <= 0) { /* refill if buf is empty */
6         rp->rio_cnt = read(rp->rio_fd, rp->rio_buf,
7                             sizeof(rp->rio_buf));
8         if (rp->rio_cnt < 0) {
9             if (errno != EINTR)
10                 return -1 ;
11         }
12         else if (rp->rio_cnt == 0) /* EOF */
13             return 0;
14         else
15             rp->rio_bufptr = rp->rio_buf; /* reset buffer ptr */
16     }
17
```

```
18      /* Copy min(n, rp->rio_cnt) bytes
        from internal buf to user buf */
19      cnt = n ;
20      if ( rp->rio_cnt < n)
21          cnt = rp->rio_cnt ;
22      memcpy(usrbuf, rp->rio_bufptr, cnt) ;
23      rp->rio_buffer += cnt ;
24      rp->rio_cnt -= cnt ;
25      return cnt ;
26 }
```

```
1  ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n)
2  {
3      size_t nleft = n; ssize_t nread ;
4      char *bufp = usrbuf;
5      while (nleft > 0) {
6          if ((nread = rio_read(rp, bufp, nleft)) < 0) {
7              if (errno == EINTR)
8                  /* interrupted by sig handler return */
9                  nread = 0;
10             else
11                 return -1;
12         }
13         else if (nread == 0)
14             break;
15         nleft -= nread;
16         bufp += nread;
17     }
18     return (n - nleft);
19 }
```

```

1  ssize_t rio_readlineb(rio_t *rp,
                          void *usrbuf, size_t maxlen)
2  {
3      int n, rc;
4      char c, *bufp = usrbuf;
5      for (n=1; n < maxlen; n++) {
6          if ((rc = rio_read(rp, &c, 1)) == 1) {
7              *bufp++ = c;
8              if (c == '\n')
9                  break;
10             } else if (rc == 0) {
11                 if (n== 1)
12                     return 0; /* EOF, no data read */
13                 else
14                     break;
15             } else
16                 return -1; /* error */
17     }
18     *bufp = 0 ;
19     return n ;
20 }

```

Standard I/O

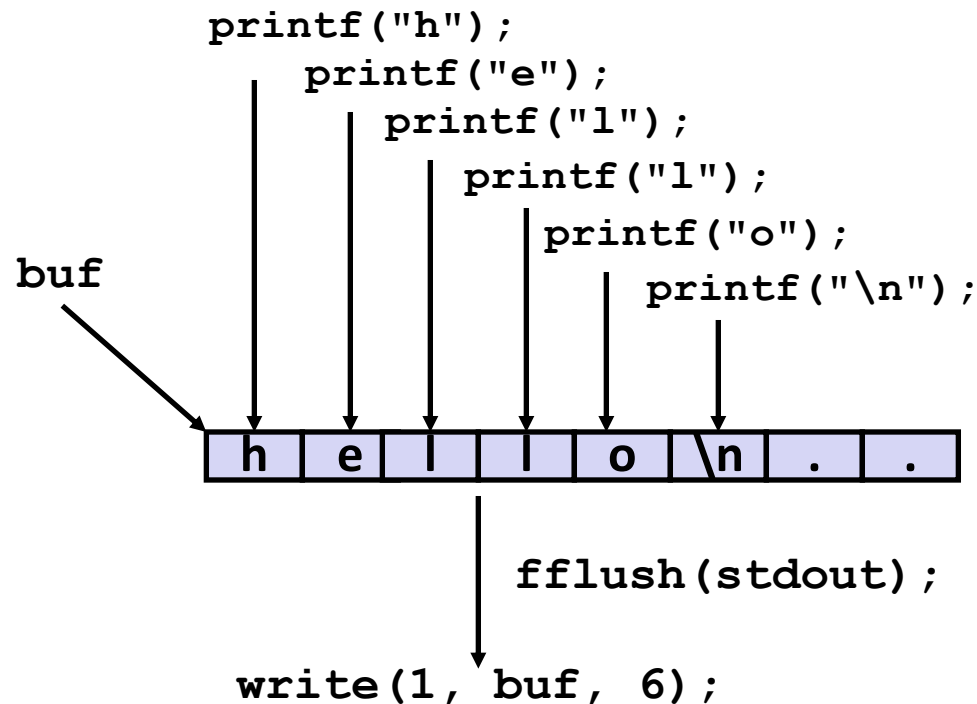
- The C standard library (`libc.so`) contains a collection of higher-level standard I/O functions
- Examples of standard I/O functions:
 - Opening and closing files (`fopen` and `fclose`)
 - Reading and writing bytes (`fread` and `fwrite`)
 - Reading and writing text lines (`fgets` and `fputs`)
 - Formatted reading and writing (`fscanf` and `fprintf`)

Standard I/O

- Standard I/O models open files as **streams**
 - Abstraction for a file descriptor and a buffer in memory.
 - Similar to buffered RIO
- C programs begin life with three open streams (defined in `stdio.h`)
 - `stdin` (standard input `fd=0`)
 - `stdout` (standard output `fd=1`)
 - `stderr` (standard error `fd=2`)

Buffering in Standard I/O

- Standard I/O functions use buffered I/O



- Buffer flushed to output fd on `"\n"` or `fflush()` call

Recap: Fork + Print + Dup2

```
tmac@intel12-pc:~/ics/exception-process$ cat fork.c
```

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    for (int i = 0; i < 2; ++i) {
```

```
        fork();
```

```
        printf("Hello ICS\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
tmac@intel12-pc:~/ics/exception-process$ ./a.out
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
tmac@intel12-pc:~/ics/exception-process$ ./a.out > tmp
```

```
tmac@intel12-pc:~/ics/exception-process$ cat tmp
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

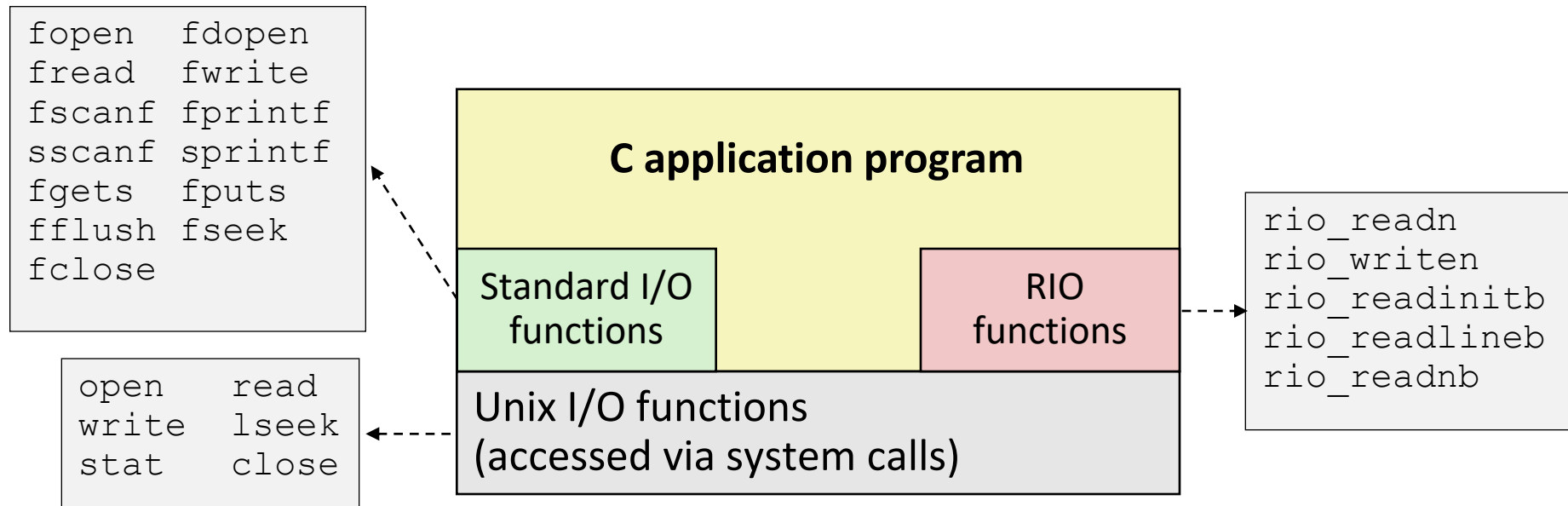
```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

Unix I/O, Standard I/O, and Robust I/O

- Standard I/O and RIO are implemented using low-level Unix I/O



- Which ones should you use in your programs?

Pros and Cons of Unix I/O

- Pros

- Unix I/O is the most general and lowest overhead form of I/O
 - All other I/O packages are implemented using Unix I/O functions.
- Unix I/O provides functions for accessing file metadata
- Unix I/O functions are async-signal-safe and can be used safely in signal handlers.

Pros and Cons of Unix I/O

- Cons
 - Dealing with short counts is tricky and error prone
 - Efficient reading of text lines requires some form of buffering, also tricky and error prone
 - Both of these issues are addressed by the standard I/O and RIO packages.

Pros and Cons of Standard I/O

- Pros:
 - Buffering increases efficiency by decreasing the number of `read` and `write` system calls
 - Short counts are handled automatically

Pros and Cons of Standard I/O

- Cons:
 - Provides no function for accessing file metadata
 - Standard I/O functions are not async-signal-safe, and not appropriate for signal handlers.
 - Standard I/O is not appropriate for input and output on network sockets
 - There are poorly documented restrictions on streams that interact badly with restrictions on sockets

Restrictions on Streams

- Input functions following output functions.
 - An input function cannot follow an output function without an intervening call to `fflush`, `fseek`, `fsetpos`, or `rewind`.
 - The `fflush` function empties the buffer associated with a stream.
 - The latter three functions use the Unix I/O `lseek` function to reset the current file position.

Restrictions on Streams

- Output functions following input functions.
 - An output function cannot follow an input function without an intervening call to `fseek`, `fsetpos`, or `rewind`, unless the input function encounters an end-of-file

Root Cause

- Standard IO uses a user-space buffer and maintains a single "current position" pointer
- This buffer is generally designed to operate in one state (read or write) at a time
 - fflush, fseek, fsetpos, or rewind reset buffer state

Socket Problem

- Sockets are sequential streams, not random-access files
 - Calling `fseek` on a socket will fail
- Solution: dual streams
 - Use `dup()` to duplicate the socket file descriptor
 - Call `fdopen()` twice to create two separate streams, one for reading and one for writing
 - Still need to use `fflush` to avoid deadlock

Standard IO with Socket

```
FILE *fpin, *fpout;  
copy_fd = dup(sockfd)  
fpin = fdopen(sockfd, "r");  
fpout = fdopen(copyfd, "w");  
  
... // fprintf(fpout, ..); fflush(fpout); fgets(fpin, ..)  
  
fclose(fpin);  
fclose(fpout); // must close both
```

Choosing I/O Functions

- General rule: use the highest-level I/O functions you can
 - Many C programmers are able to do all of their work using the standard I/O functions

Choosing I/O Functions

- When to use standard I/O
 - When working with disk or terminal files
- When to use raw Unix I/O
 - Inside signal handlers, because Unix I/O is async-signal-safe.
 - In rare cases when you need absolute highest performance.
- When to use RIO
 - When you are reading and writing network sockets.
 - Avoid using standard I/O on sockets.